



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure
E – Environment
A – Actuators
S – Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width: number of columns
self.height: number of rows
self.food_positions: number of food items to place
self.opponents: number of enemies
self.agent_pos: agent position
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent Environment

There's only one player agent but with the `self.opponent` it stores the player agents. these agents plays independently so it affect to the other player's performance.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A Percept Sequence is the complete history of all percepts that an agent has received from the environment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

sensors

`get_percept()` collects information from the environment and sends it to the agent.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable

The agent receives only the information returned by `get_percept()`. It cannot directly observe every detail of the environment.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure

When the agent hits a wall `self.score -= 5` the score changes. Score measures how well the agent performs. So the Performance Measure is updated.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Because intelligent agents should be evaluated based on their effect on the environment, not how much computation they perform. If performance measured CPU usage or thinking time, an agent could exploit the metric without solving the task. The goal is to maximize success in the environment.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes more partially observable. The traps exist but the agent cannot detect them. Therefore the agent must make decisions with incomplete information.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The sensor gives advance warning about nearby toxic traps. The agent can avoid dangerous cells, increasing its expected utility and improving rational decision-making. A rational agent chooses the action expected to produce the best outcome.

Without the sensor: Walk blindly -> Hit trap -> Lose points
With the sensor: Detect toxin -> Avoid trap -> Higher score

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World exploit is when an agent repeatedly performs unnecessary actions (such as continuously cleaning an already clean room) simply to increase its performance score due to a poorly designed performance measure.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING